

Padrões e Ferramentas para Documentação de APIs

Objetivo da Aula

Conhecer padrões e boas práticas na documentação de APIs e utilizar ferramentas integradas para documentar o projeto NodeJS.

Apresentação

Uma API define um conjunto de serviços que tem como objetivo a utilização por terceiros, de uma forma independente de plataforma, bastando que seja aderente ao modelo de comunicação adotado. Com o acesso por pessoas que não fazem parte do time interno de desenvolvimento, a documentação se torna um componente fundamental para utilização da API.

Devemos entender como criar uma documentação coesa e completa, que dê todo o suporte necessário ao desenvolvedor. Veremos algumas boas práticas na construção dos documentos, bem como alguns padrões amplamente utilizados no mercado para a especificação desses documentos. Também veremos algumas ferramentas para especificar a documentação, em especial o Swagger, que utilizaremos em um projeto de API com NodeJS.

1. Boas Práticas na Documentação de APIs

Ao documentar uma API, precisamos oferecer as informações necessárias para que um desenvolvedor independente possa utilizá-la. Devemos considerar os seguintes componentes na documentação:

- Serviços e **Endpoints**;
- Tipos e **Entidades**;

- Argumentos e Retornos; e
- Tutoriais de Exemplo.

Uma documentação deve ser **consistente**, mapeando todos os elementos de uma API, **coesa**, evitando fornecer informações fora do contexto, e **completa**, cobrindo todos os aspectos necessários para utilização da API. Um artifício que pode se tornar um bom reforço para a documentação é a criação de um aplicativo de demonstração de uso, ou **demo**, implementado em diversas linguagens.

É importante a centralização de serviços por área de interesse, agrupando por domínio ou objetivo. Em geral, temos uma navegação lateral para tópicos principais, e uma navegação interna baseada em **links** no interior da página.

As mensagens de erro devem estar presentes na documentação, fornecendo explicações detalhadas e métodos de tratamento. Sempre teremos que usar exemplos em várias linguagens, já que uma API visa a interoperabilidade.

Uma boa **introdução**, bem sólida e detalhada, será bastante útil para que o desenvolvedor externo entenda o contexto da API. Em geral, procuramos fazer testes com **equipes independentes**, para que possamos certificar o sistema.

Hoje em dia é muito comum disponibilizar **snacks** de código, que podem ser utilizados pelo desenvolvedor para testar os códigos antes de começar a criar um sistema. Um **ambiente de testes**, sem movimentações financeiras reais, pode ser disponibilizado para efetuar os testes sem encargos, e sem correr o risco de alterar os dados no ambiente de produção.

Por se tratar de um produto voltado para a interoperabilidade, diversos outros sistemas utilizarão nossa API. O **versionamento** da documentação, bem como de cada estrutura de dados ou serviço, é essencial para que os desenvolvedores encontrem informações consistentes, sem elementos com obsolescência.



Destaque

Em geral, quando um serviço tende a ser excluído em uma versão futura, ele deve ser marcado como deprecated, ou obsoleto.

Buscamos trabalhar com hierarquias de busca pequenas, ou seja, queremos uma documentação o mais **plana** possível, com navegação lateral que se abre, no máximo, em

um ou dois níveis. De forma geral, as informações de um serviço específico devem ser apresentadas em uma só página.

O uso de **cores** e **simbologias** padronizadas é muito importante para permitir uma identificação rápida dos tipos de componentes. Toda documentação deve ser intuitiva, permitindo encontrar as informações de forma rápida. Finalmente, a utilização de algumas **tabelas**, com descrições rápidas e links para acesso direto, aumentam a agilidade na navegação.

2. O Padrão OpenAPI

Uma documentação de API, dentro de padrões consistentes, adotados pelo mercado de forma geral, pode ser utilizada em diversas fases do ciclo de vida, como no projeto, direcionando os programadores, nos testes, com a construção de **servidores simulados** em ferramentas como **Insomnia**, e na implantação, para que os desenvolvedores externos saibam como acessar os serviços.

Apesar de ser interessante oferecer uma boa quantidade de exemplos, com base em diversas linguagens, a descrição de serviços e protocolos deve utilizar uma abordagem **agnóstica**, sem se voltar para uma linguagem específica. Para que a documentação tenha sucesso, é importante que ela seja usada pela maior quantidade de desenvolvedores, em diferentes plataformas.

O padrão **OpenAPI** permite gerar uma documentação nesse perfil, utilizando sintaxe simples, com base em **JSON** ou **YAML**. É um padrão aberto, com ampla aceitação pelo mercado, e que foi iniciado como especificação do Swagger. Na listagem seguinte temos o início da documentação no formato YAML.

```
openapi: 3.0.0
info:
  title: API para gerenciamento de livros
  version: 0.0.2
paths:
  /livros/{id}:
    get:
      ...
```

Um documento no padrão OpenAPI inicia com a **versão** da especificação que será utilizada, seguido de um objeto de **informações gerais** da API. Em seguida, temos a divisão **paths**, com os **endpoints**, e cada **método HTTP** de acesso no nível seguinte. No formato YAML, o código deve ser endentado e seguir um padrão de espaçamento fixo. Em geral, o formato JSON é mais utilizado, como no exemplo seguinte, com o detalhamento de uma rota, parte de um documento no formato OpenAPI.

```
{
  "/pets": {
    "get": {
      "description": "Returns all pets from the system",
      "responses": {
        "200": {
          "description": "A list of pets.",
          "content": {
            "application/json": {
              "schema": {
                "type": "array",
                . . . . .
```



Destaque

Você pode conferir a especificação completa do padrão OpenAPI no endereço:
<https://spec.openapis.org/oas/v3.1.0>.

3. Ferramentas de Documentação

Existem diversas ferramentas para a documentação de APIs, normalmente baseadas em padrões como **Open API** ou **API Blueprint**. Muitas delas podem ser utilizadas na Web e, em geral, além de permitir a **documentação** de uma API já existente, podem ser utilizadas para modelar e **gerar** as APIs.

A ferramenta **Apiary** (apiary.io) permite a criação de APIs de forma rápida, com base na documentação. Trata-se de um modelo Web de desenvolvimento **colaborativo**, onde as equipes compartilham especificações, dentro de um ciclo completo, que envolve projeto, especificação e geração da API.

Para definir as chamadas da API que será gerada, **Apiary** utiliza uma sintaxe denominada **API Blueprint**. Essa sintaxe visa oferecer uma linguagem de alto nível para descrição de serviços, em um formato que seja acessível para todos os envolvidos no ciclo de vida da API.

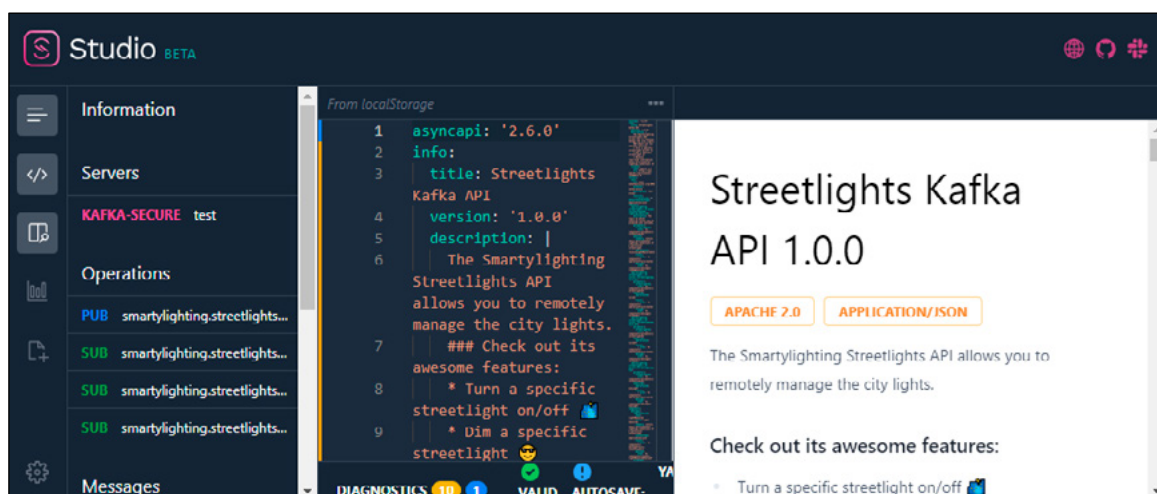
Um pequeno exemplo de API Blueprint pode ser observado a seguir. Note como o formato é bastante intuitivo, com a descrição de chamadas, valores de retorno e comentários em geral.

```
# Blog Posts [/posts]

## Retrieve All Posts [GET]
+ Response 200 (application/json)
  + Attributes (array[Blog Post])
```

Para a comunicação por mensagerias, dentro da arquitetura **Event Driven**, o padrão **AsyncAPI** é uma adaptação da Open API que se integra com o ambiente assíncrono. Você pode experimentar esse padrão no playground, ambiente de testes e estudos da plataforma, no endereço <https://studio.asyncapi.com/>.

Figura 1: AsyncAPI Studio – playground – captura de tela do Chrome



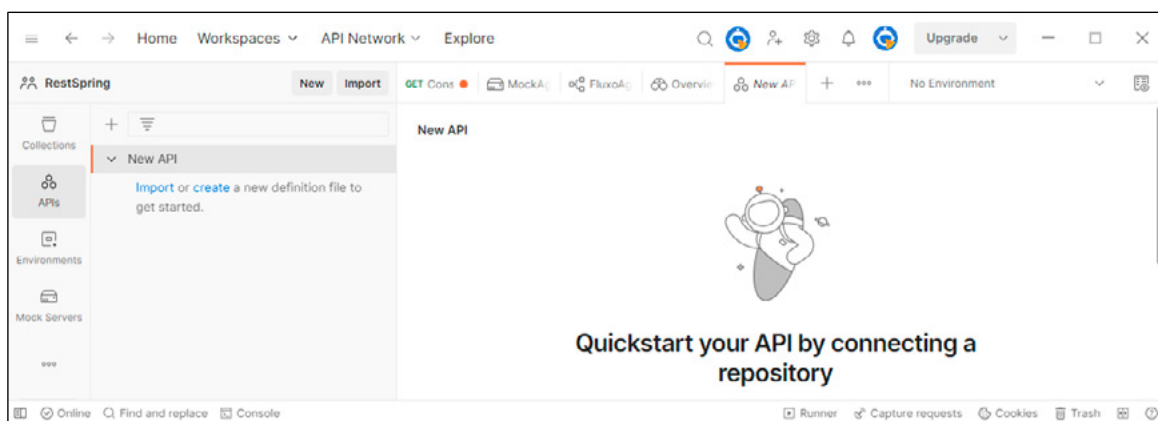
Fonte: Elaboração própria.

Outra ferramenta interessante é a **API Designer** da MuleSoft. Ela permite a documentação durante todo o ciclo de vida da API, com suporte a formatos como RAML, Open API, AsyncAPI e GraphQL.

Através do API Designer é possível criar uma API com base em **fragmentos** reutilizáveis, como gerenciadores de segurança e entidades comuns. Também completa a especificação da API por meio de **sugestões** sensíveis ao contexto.

Para os testes da API, é possível utilizar seu **servidor simulado**, deixando os serviços disponíveis para as equipes de desenvolvimento, mesmo antes de sua concepção. Em termos de servidores simulados, ou **mock servers**, no Postman podemos simular nossa API, bastando importar a especificação Open API dela, tanto no formato YAML quanto JSON. O mais interessante é que o mock server pode ser integrado no gerenciador de fluxos de execução do Postman, permitindo simular ambientes de grande complexidade com poucos recursos.

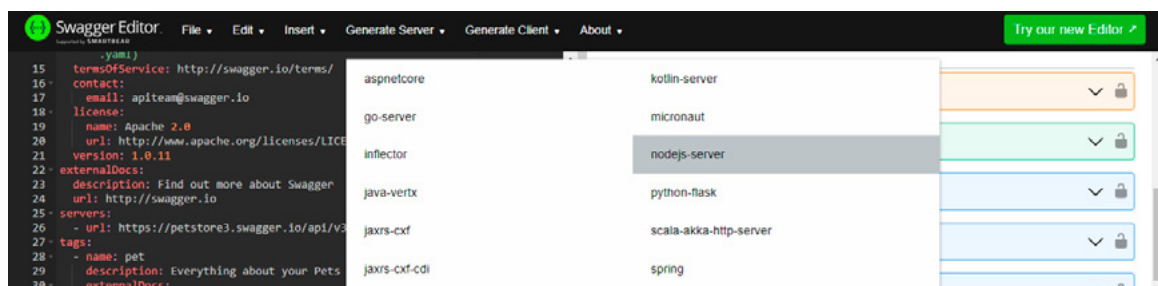
Figura 2: Divisão de modelagem e criação de API – Captura de tela do Postman



Fonte: Elaboração própria.

Uma opção online, gratuita e de grande utilização por desenvolvedores é o Swagger Editor online, disponível no endereço **<https://editor.swagger.io/>**.

Figura 3: Swagger Editor online – Captura de tela do Chrome



Fonte: Elaboração própria.

No editor online do Swagger, a tela é dividida em duas partes, uma com o código YAML ou JSON, e outra com a visualização da documentação em uma tela Swagger UI. A visualização é alterada em tempo real, à medida em que digitamos a documentação de nossa API.

Além de facilitar a visualização da documentação final, mesmo durante sua especificação, o editor permite gerar código para servidores e clientes da API. Entre as muitas opções disponíveis está o NodeJS, o que significa que podemos detalhar o Web Service RESTful na ferramenta, baixar o projeto NodeJS gerado, e apenas completar as funcionalidades no código.

4. Uso de Swagger no NodeJS

O ambiente do **NodeJS** tem uma integração muito simples com o **Swagger** quando utilizamos o framework **Express**. Inicialmente, vamos criar um diretório vazio, abrir no VS Code, e digitar os comandos seguintes.

```
npm init -y  
npm i express swagger-ui-express
```

Após incluir as dependências, crie o arquivo **index.js**, e utilize o código que é apresentado na listagem seguinte.

```
const express = require('express')  
const swaggerUi = require('swagger-ui-express')  
const swaggerDocument = require('./swagger.json')  
  
const app = express();  
app.use('/api-docs', swaggerUi.serve,  
        swaggerUi.setup(swaggerDocument));  
  
app.listen(3000);
```

Aqui configuramos um servidor **Express** comum, e adicionamos uma rota em **api-docs**, a qual utiliza o fornecedor de conteúdo do **SwaggerUI**. Para configurar o servidor de conteúdo, foi utilizado o método **setup**, recebendo um arquivo no formato **JSON**, como pode ser observado na definição de **swaggerDocument**.

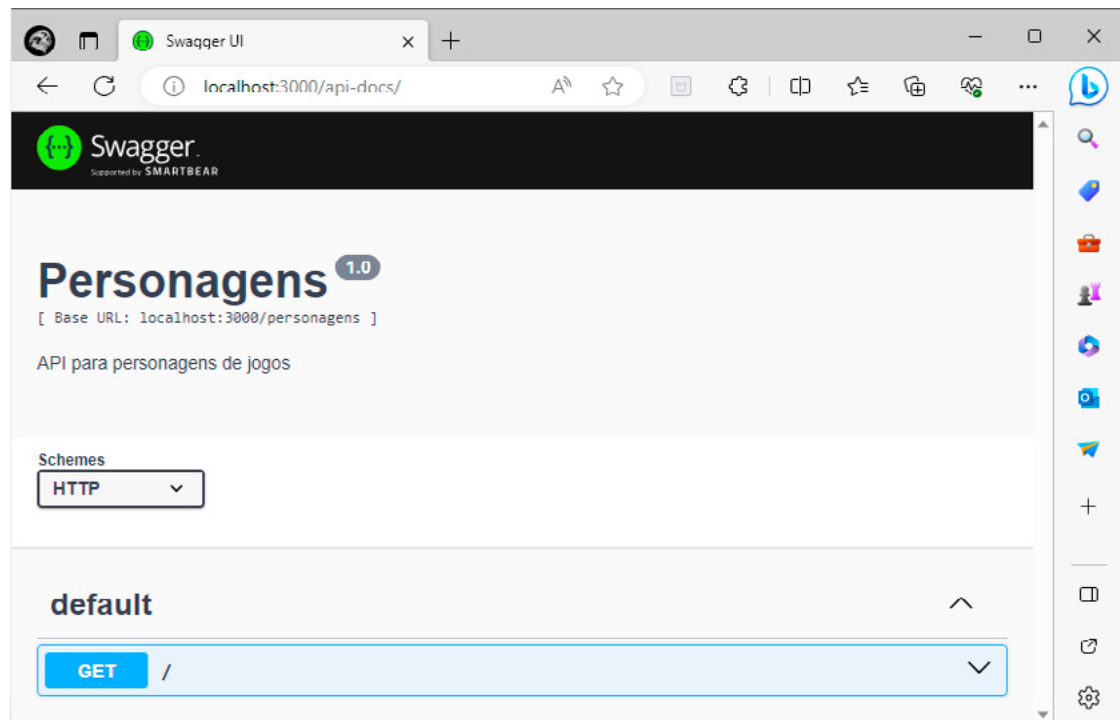
Agora precisamos do arquivo **swagger.json**, com o conteúdo seguinte.

```
{
  "swagger": "2.0",
  "info": {
    "title": "Personagens",
    "description": "API para personagens de jogos",
    "version": "1.0"
  },
  "host": "localhost:3000", "basePath": "/personagens",
  "schemes": [ "http" ],
  "paths": {
    "/": {
      "get": {
        "description": "Consulta aos personagens.",
        "operationId": "LIST", "responses": {}
      }
    }
  }
}
```

De acordo com a listagem, temos a **versão** do Swagger, seguido de algumas informações, no campo **info**. Em seguida, temos a definição de **servidor**, **porta** e caminho de **base**, e no schema HTTP, foi incluída apenas a requisição **GET**.

Com base nas operações, já podemos iniciar o servidor, através da execução do comando **node index**. Após deixar o servidor em execução, use o endereço **http://localhost:3000/api-docs** e veja o resultado.

Figura 4: Apresentação da documentação do projeto via Swagger UI – Captura de tela do Edge



Fonte: Elaboração própria.

Podemos observar como o **swaggerUI** utiliza as informações do arquivo JSON, e constrói uma interface visual bem-organizada. São utilizadas cores, para diferenciar o tipo de requisição, de acordo com o padrão de mercado. Agora, crie o arquivo **rota-dummy.js**, utilizando o conteúdo seguinte.

```
const express = require('express')
const rota = express.Router();

rota.get("/", (req, res)=>{});
rota.get("/:id", (req, res)=>{});
rota.post("/", (req, res)=>{});
rota.put("/:id", (req, res)=>{});
rota.delete("/:id", (req, res)=>{});
module.exports = rota;
```

Aqui, definimos apenas um roteador que oferece todos os acessos de uma **API REST**, mas sem a implementação das respostas. Ele será utilizado para demonstrar a **documentação automática** com Swagger. Instale a dependência do **Swagger Autogen**.

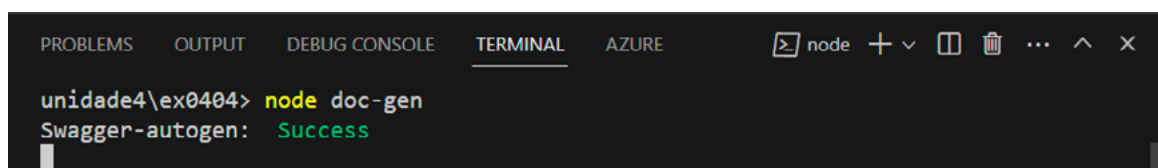
```
npm install swagger-autogen
```

Crie o arquivo **doc-gen.js**, utilizando o conteúdo seguinte.

```
const swaggerAutogen = require('swagger-autogen')( )
const outputFile = './swagger_output.json'
const endpointsFiles = ['./rota-dummy.js']
swaggerAutogen(outputFile, endpointsFiles).then(() => {
  require('./index.js')
})
```

Para trabalhar com o gerador automático, tudo que precisamos é do nome do arquivo **JSON** que será gerado e de um vetor com todos os arquivos com os **roteadores Express**. Ao final da geração, o servidor é iniciado, devido à inclusão do arquivo **index.js**, mas como ainda não temos uma rota para a documentação gerada, não será possível observá-la. Execute o gerador com o comando **node doc-gen**.

Figura 5: Execução do gerador automático do Swagger - captura de tela do VS Code



Fonte: Elaboração própria.

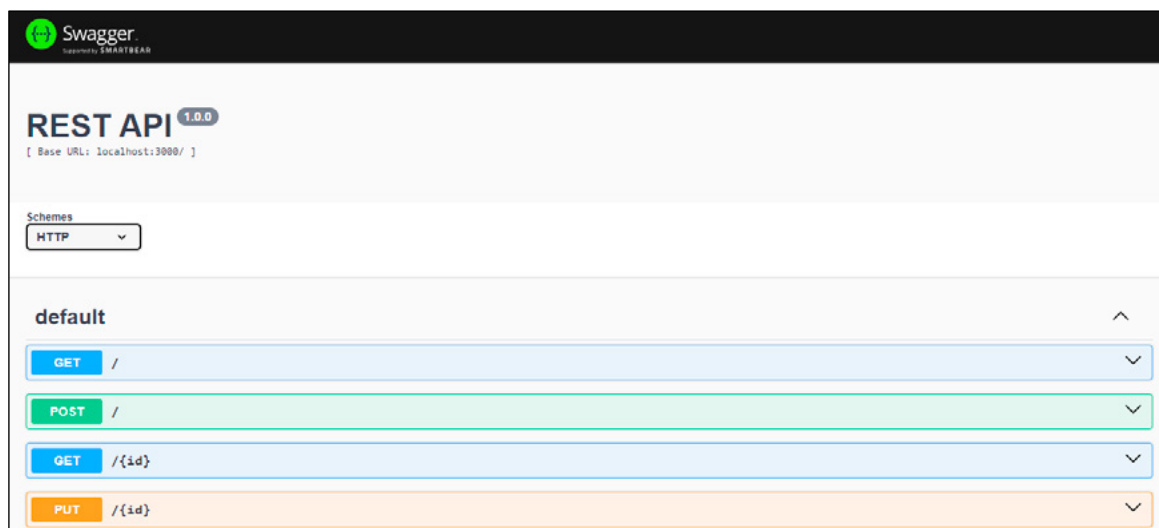
Para trabalhar com a nova documentação, vamos alterar o arquivo **JSON** lido por **swaggerDocument**, no arquivo **index.js**.

```
const express = require('express')
const swaggerUi = require('swagger-ui-express')
```

```
const swaggerDocument = require('./swagger_output.json')
const app = express();
app.use('/api-docs', swaggerUi.serve,
    swaggerUi.setup(swaggerDocument));
app.listen(3000);
```

Após salvar a alteração, execute novamente o gerador, deixando o servidor em execução, acesse o endereço **http://localhost:3000/api-docs**, e veja como ficou a documentação gerada.

Figura 6: Documentação gerada pelo Swagger de forma automática – captura de tela do Edge



Fonte: Elaboração própria.

Teremos uma documentação básica, apenas com a indicação das chamadas para o roteador, mas você pode alterar manualmente o arquivo JSON e executar o servidor apenas, através do comando **node index**.

Considerações Finais da Aula

Aqui, pudemos compreender os requisitos ideais para a construção de uma boa documentação, como o uso de padrões, hierarquia mínima de busca, snippets de código, ambientes de teste e fornecimento de tutoriais. Também pudemos ver o padrão de especificação Open API, para a documentação de APIs REST, com escrita nos formatos JSON e YAML.

Não poderíamos deixar de lado a preocupação com versionamento, tanto no formato da documentação quanto para a API em si. Nesse contexto, vimos que um serviço que tende a ser descartado deve ser marcado como deprecated, para que os desenvolvedores possam buscar alternativas à sua utilização.

Conhecemos algumas ferramentas para documentação de APIs e vimos que ela pode ser utilizada não só para instruir as equipes de terceiros, mas também para gerar o código inicial do sistema. Por fim, conhecemos melhor a ferramenta de documentação Swagger, com a integração dela no projeto NodeJS, para uma API REST baseada em Express, e pudemos perceber como a geração dos documentos foi simplificada.

Materiais Complementares



Site

Open API Initiative

Open API.

O site oficial do Open API oferece a especificação completa do padrão, além de diversas informações gerais sobre ferramentas.

Link para acesso: <https://www.openapis.org/> (acesso em: 14 set. 2023.)



Artigo

Conheça 5 editores de design para sua API

2016, Mundo API.

O site apresenta e compara 5 boas ferramentas para documentar e gerar APIs, com os endereços dos fornecedores de cada uma.

Link para acesso: <https://mundoapi.com.br/tutoriais/editores-de-design-de-apis/> (acesso em: 14 set. 2023.)



Site

Swagger – API Development for Everyone

Swagger.

O site oficial do Swagger oferece diversas informações sobre a ferramenta, incluindo a utilização de Swagger Codegen, Swagger Editor e Swagger UI.

Link para acesso: <https://swagger.io/> (acesso em: 14 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. ISBN 9788565837484. Disponível em: <https://biblioteca.sophia.com.br/termi->

nal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidadePaginas%3d1%26codigoRegistro%3d92164%2392164&i=8. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. ISBN 9788536533100. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. ISBN 9786558110217. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 14 set. 2023.